```
$ npm i -g birc-generator
```

Springboot4 實戰

// 從 npm 裝產生器，做到一本 Book 的真 CRUD

create
開專案

make
CRUD 骨架

migrate + seed
Flyway 建表、Seeder 塞列

profiles
dev / prod

$ cat syllabus.md

今天會做完什麼

01 / 安裝
npm i -g birc-generator，確認 birc 能跑

02 / 開專案
birc create bookstore --yes

03 / Book CRUD
make + migrate + seed，真的打得到 API

04 / 加關聯
Author、FK、MapStruct 帶出 authorName

05 / 改版
ValidGroup、Spotless、record、上傳、Docker、CORS

先對齊工具版本

要有	版本	用來幹嘛
Node	18 以上	跑 `birc`
Docker	能跑 `docker compose`	只起 MySQL
Java	25（之後 Gradle 會抓 toolchain）	Spring Boot 4.0.5

```
CHECK

node -v
docker compose version
```

Java 現在不用先裝齊。第一次 `./gradlew bootRun` 會靠 toolchain 補。

PART 01

從 npm 裝 birc

CLI 叫 birc，套件名是 birc-generator

為什麼不手抄舊專案？

以前開 BIRC 後端，常是把舊 Controller 改名再貼一次。套件路徑改到一半、DAO 繼承寫錯、例外類從 starter 抓來用，之後還得再修。

create
多模組 Spring Boot 4 骨架

make
一組 CRUD 該有的層

安裝入口只有一個：`npm i -g birc-generator`

安裝

```
npm i -g birc-generator
birc -v
birc -h
```

WSL / macOS 權限

裝不進去時，先確認你用 nvm 或自己的 prefix，不要習慣性 sudo。

更新用 `birc update`，等價 `npm i -g birc-generator@latest`。

指令地圖（今天會用到的）

指令	做什麼
`birc create`	開多模組專案
`birc make`	Entity + DAO + Mapper + DTO + Service + Controller
`birc make:model --migration`	Entity + DAO，順便生 Flyway
`birc make:migration`	只生 SQL：`create_` / `add_` / `change_`
`change_{欄}_on_{表}_table`	改型別：先改 Entity，再出 `MODIFY COLUMN`
`--soft-delete`	`make` 時加 `@SoftDelete` + `deleted_at`
`birc make:seeder`	只生 Java Seeder
`birc migrate`	真的跑 `flywayMigrate`，只建表
`birc seed`	執行 Seeder，才有示範資料

PART 02

開一個 bookstore

create 一定是多模組，沒有單模組選項

建專案

挑一個空目錄（不要建在別人的 git repo 裡）：

```
mkdir -p ~/birc-lab && cd ~/birc-lab
birc create bookstore --yes
cd bookstore
```

--yes
不問了。docker / log4j2 / ValidGroup / Spotless 預設勾

package
bookstore → tw.edu.ntub.birc.bookstore

想自己勾功能就拿掉 `--yes`，互動選。課堂為了同步，用 `--yes`。

資料夾長這樣

```
bookstore/
  Application.java
  build.gradle / settings.gradle / gradlew
  .bircrc.json          ← birc 找專案根用這個
  .env / .env.example   ← create 有勾 docker 才會複製
  docker-compose.yml
  docker-compose.prod.yml
  modules/bookstore-config
  modules/bookstore-database-config
```

API 在根專案。Entity / DAO 在 `*-database-config`。Security、CORS 在 `*-config`。

先起資料庫，不要整包 compose up

容器只跑資料庫。App 用本機 `bootRun`，比較快、也好 debug。

```
docker compose up -d db
set -a && source .env && set +a
```

PowerShell：

```
docker compose up -d db
Get-Content .env | ForEach-Object {
  if ($_ -match '^\s*([^#][^=]*)=(.*)$') {
    Set-Item -Path "Env:$($matches[1].Trim())" -Value $matches[2].Trim().Trim('"')
  }
}
```

`docker compose` 會自己讀專案裡的 `.env` 來起 MySQL。`bootRun` 不會。沒 `source` 時 JVM 用 yml 後備值：庫名 `app`、帳號 `root`、密碼空白，跟容器裡的庫名和隨機帳密對不上。

1. DTO 改成 record

以前常見 Lombok `@Data` class。現在 CreateRequest / Response 是 Java record。

	class + setter	record
可變性	呼叫端可改欄位	建立後就不能改
樣板	getter/setter/equals 自己產	編譯器給
定位	容易跟 Entity 長得太像	一眼看出這是 API 語言

Entity 仍是 class（JPA 要無參建構子與可變欄位）。record 留給進出 API 的資料。

.env 怎麼接到 Spring

`.env` 裡這些鍵，Spring 用 `${}` 讀：

```
DB_URL=jdbc:mysql://127.0.0.1:3306/bookstore
DB_USERNAME=bookstore_user
DB_PASSWORD=...
CORS_ALLOWED_ORIGIN_PATTERNS=
```

`application.yml`：

```yaml
spring:
  datasource:
    url: ${DB_URL:jdbc:mysql://localhost:3306/app}
    username: ${DB_USERNAME:root}
    password: ${DB_PASSWORD:}
app:
  cors:
    allowed-origin-patterns: ${CORS_ALLOWED_ORIGIN_PATTERNS:}
```

容器裡的 App 不讀你筆電上的 `.env` 檔，讀 compose 的 `environment:`。所以 compose 才要再寫一次 `DB_URL: jdbc:mysql://db:3306/${DB_DATABASE}`（host 是服務名 `db`，不是 127.0.0.1）。

PART 03

生一本 Book 的 CRUD

骨架用 make，表用 Flyway，示範列用 Seeder，不要靠 ddl-auto:update

一次生齊：birc make

指令	會寫出什麼
`birc make Book`	Entity、DAO、Mapper、DTO、Service、Controller
`birc make:model Book --migration`	Entity、DAO，加上 Flyway
`birc make Book --migration --seed`	各層 + Flyway SQL + `BookSeeder.java`
`birc make:controller Book`	只有 Controller

注意

`make:model --controller` 只加 Controller，不加 Service / Mapper。

Controller 會 import 不存在的 `BookService`，編不過。

課堂指定指令

一行把各層、建表 SQL、Seeder 都生出來：

```
birc make Book --fields title:String,isbn:String,price:BigDecimal,publishedAt:LocalDate --migration --seed
```

注意沒有 `author`。作者等一下要做成**另一張表的關聯**，現在先不要生成字串欄位，否則之後會跟 `Author` 撞名。

--fields
寫你自己的欄位。有它就是完整產出：欄位、CRUD、@Column

--migration
讀 Entity 產建表 SQL

--seed
產 BookSeeder.java，不是 INSERT

`--fields` 或 `--example` 有一個就是完整產出，不必兩個一起加。差別只在欄位是你指定的，還是範本的 `id` / `name`。

這行只是「產生檔案」。要進資料庫還要 `birc migrate` 和 `birc seed`。

會寫出哪些檔

層	路徑（精簡）
Entity / DAO	`modules/.../databaseconfig/entity/Book.java`、`BookDAO.java`
DTO	`dto/BookCreateRequest.java`、`BookResponse.java`（**record**）
Mapper	`mapper/BookMapper.java`（MapStruct）
Service	`BookService` + `BookServiceImpl extends BaseServiceImpl`
Controller	`/api/books`
Flyway	`src/main/resources/db/migration/V1__create_book_table.sql`
Seeder	`seeder/BookSeeder.java`（`--seed` 才有）

表名是 snake_case 的 entity：`book`。URL 是 `/api/books`（名稱後面加 s）。

`--migration` 直接讀 Entity

`--migration` 直接讀 Entity Java 檔，自動產生對應的 Flyway SQL，不用手改欄位。

Entity 裡的 `@Column`、型別、長度限制都會反映在 SQL 裡。**要修改既有表結構，請新增一筆遷移**（例如 `add_...`、`change_...`），不要覆寫既有 migration.

```
# 一次產出 Entity + Migration SQL + Seeder（檔案而已）
birc make Book --fields \
  title:String,isbn:String,price:BigDecimal,publishedAt:LocalDate \
  --migration --seed --public-read
```

SQL 裡只有 `CREATE TABLE`。示範資料不會寫成 `INSERT`，那是 Seeder 的事。

套用遷移

`application.yml` 裡 `jpa.hibernate.ddl-auto` 是 **validate**。沒跑 Flyway，bootRun 會起不來。

```
# 同一個 shell，.env 還在
birc migrate
```

這條會呼叫專案裡的 Gradle wrapper `flywayMigrate`，不是只印指令。**只建表，不塞資料。**

ROLLBACK？

Flyway Community 做不到 rollback。

`birc migrate:rollback` 只會告訴你：再寫一筆往前的遷移。

課堂打壞表：`birc migrate:reset --force`（flywayClean + migrate）。

reset 之後表會回來，示範列不會，要再跑一次 `birc seed`。

示範資料走 Seeder，不走 SQL

`--seed` 產的是 Java，不是 `INSERT`。有 Entity 就照欄位帶 `setXxx`：

```java
public class BookSeeder {
    public void run() {
        Book book = new Book();
        book.setTitle("Domain-Driven Design");
        book.setIsbn("9780321125217");
        // price / publishedAt ...
        dao.save(book);
    }
}
```

`birc seed` 才會真的執行它。

表結構的唯一來源是 Flyway；示範列的唯一來源是 Seeder。兩邊不要混。

完整流程是三步

```
birc make Book --fields ... --migration --seed   # 只產檔案
birc migrate                                     # 只建表
birc seed                                        # 才真的塞示範資料
```

make
寫檔案，什麼都還沒進 DB

migrate
schema 進 DB

seed
資料列進 DB

最常見的卡點：跑完 `migrate` 就去 `GET /api/books`，拿到空陣列以為壞了。少的是 `birc seed`。

事後才想補 Seeder：`birc make:seeder BookSeeder`（命名跟 Laravel 一樣帶後綴）。

課堂必改：否則 API 全是 401

SecurityConfig 是 `birc create` 產出來的，預設 `anyRequest().authenticated()`。沒接 auth 模組時沒有任何東西能通過它（沒登入端點、沒 JWT filter），所以沒列進去的路徑一律鎖死。

不用自己寫，讓產生器產：

```
birc make:security Book
```

產出 `src/main/java/.../config/BookSecurityCustomizer.java`：

```
@Component
public class BookSecurityCustomizer implements SecurityCustomizer {

    @Override
    public List<String> publicPaths() {
        return List.of("/api/books", "/api/books/**");
    }
}
```

兩個關鍵字不能少

implements SecurityCustomizer
SecurityConfig 用 `ObjectProvider<SecurityCustomizer>` 收集實作，型別不符就永遠收不到。

@Component
沒有它就不會被 Spring 掃到，收集到的集合是空的。

不要自己宣告一個新介面

自己寫一個 `interface BookSecurityCustomizer`（不 implements、沒 `@Component`）無效：型別跟專案裡的 `SecurityCustomizer` 無關，收不到，`authenticated()` 照樣鎖死全部。而且沒有任何 log 或例外，只會安靜地一直回 401。

另一種寫法：建 Entity 時加 `--public-read` 一起產，它會多掛 Controller 的 `@PreAuthorize`；`make:security` 只動 filter 層。

啟動

```
./gradlew bootRun
```

Windows：`.\gradlew.bat bootRun`

等看到 Tomcat 8080。第一次會下載 Gradle 與依賴，慢慢等。

> **不要**這時候 `docker compose up` 把 app 也建進映像。Dockerfile 會在容器裡跑 Gradle，課堂會卡住。

真的打 CRUD

```
curl -s -X POST http://localhost:8080/api/books \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "Domain-Driven Design",
    "isbn": "9780321125217",
    "price": 1800,
    "publishedAt": "2003-08-30"
  }'
```

```
curl -s http://localhost:8080/api/books          # 先看 seed 的資料在不在

curl -s http://localhost:8080/api/books/1

curl -s -X PUT http://localhost:8080/api/books/1 \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "Domain-Driven Design",
    "isbn": "9780321125217",
    "price": 1800,
    "publishedAt": "2003-08-30"
  }'

curl -s -X DELETE http://localhost:8080/api/books/1
```

請求 / 回應是 record，不是 Entity

```java
public record BookCreateRequest(
        @NotNull(groups = {ValidGroup.Create.class, ValidGroup.Update.class})
        String title,
        // isbn, price, publishedAt...
) {}

public record BookResponse(
        Long id,
        String title,
        String isbn,
        BigDecimal price,
        LocalDate publishedAt
) {}
```

Controller 只轉 DTO。Entity 不出 API。這是這次改版要記住的邊界。

CRUD 實際跑在 BaseServiceImpl

`BookServiceImpl` 幾乎是空的。create / getById / update / deleteById 寫在 `BaseServiceImpl`：

1. `mapper.toEntity(request)` → `dao.save`
2. 找不到丟 `NotFoundException`（專案自己的 `ProjectException`）
3. update 用 MapStruct `updateEntity`

額外業務（例如依 ISBN 查重）寫在 `BookService` / `BookServiceImpl`，不要重寫那四個 CRUD。

PART 03-B

擴充：Author 與 MapStruct

作者表、FK、改欄、軟刪、Optional 與 @Mapping

建 Author CRUD

```
birc make Author --fields name:String,birthYear:Integer,nationality:String --migration --seed --public
```

--migration
讀 Entity 產建表 SQL（Author 沒有關聯，就是單純一張表）

--seed
產 AuthorSeeder.java，裡面是 setXxx

```
birc migrate   # 建 author 表
birc seed      # 塞村上春樹、東野圭吾
```

Seeder

```
author.setName("村上春樹");
author.setBirthYear(1949);
author.setNationality("日本");
```

`make Author` 也生了 Controller，所以多了 `/api/authors`。要打它就把 `/api/authors`、`/api/authors/**` 一起加進 `PUBLIC_PATHS`，不然一律 401。

給 Book 表加 FK 欄位

```
birc make:migration add_author_id_to_book_table
birc migrate
```

欄位名是 `{ref}_id`、而且 `{ref}` 的表找得到時，才會一併 `ADD CONSTRAINT ... FOREIGN KEY`。

所以順序很重要：`author` 表要先建好。表還不存在時只會給你一個 `BIGINT`，不加 constraint。

改 Book Entity 加關聯

```java
@Entity
@Table(name = "book")
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id")
    private Author author;

    // ... 其他欄位
}
```

`@ManyToOne` 建立多對一關聯，`.LAZY` 避免預載入。

改 BookResponse 加 authorName

```java
public record BookResponse(
    Long id,
    String title,
    String authorName,  // ← 新增
    String isbn,
    BigDecimal price,
    LocalDate publishedAt
) {}
```

birc 不會自己改 record，要手動加 `authorName` 欄位。

改 BookMapper 加 @Mapping

```java
@Mapper(componentModel = "spring")
public interface BookMapper {
    @Mapping(source = "author.name", target = "authorName")  // ← 讀出來：帶作者名稱
    BookResponse toResponse(Book book);

    @Mapping(target = "author", ignore = true)               // ← 寫進去：先不處理
    Book toEntity(BookCreateRequest request);
}
```

帶出 `authorName` 就走這條：宣告在 Mapper，Service 不用寫程式。

但 `ignore = true` 表示 POST 設不了作者——新建的書永遠沒有 Author。下一頁補起來。

POST 怎麼指定作者

前端只會傳一個 id，不會傳整個 Author 物件。兩邊各改一處：

① DTO 收 authorId

```
public record BookCreateRequest(
    String title,
    // isbn, price, publishedAt...
    Long authorId   // 不填就是沒作者
) {}
```

② Mapper 換成 Author

```
@Mapping(source = "authorId", target = "author")
Book toEntity(BookCreateRequest request);

default Author toAuthor(Long id) {
    if (id == null) return null;
    Author a = new Author();
    a.setId(id);        // 只要 FK
    return a;
}
```

不用去查 `author` 表：寫入 `book` 只需要 `author_id` 這個值。傳不存在的 id 會被 FK constraint 擋下來，想先回 400 就自己在 Service 檢查一次。

改 BookDAO 加 @EntityGraph

```
public interface BookDAO extends BaseDAO<Book> {
    @EntityGraph(attributePaths = {"author"})
    List<Book> findAll();

    @EntityGraph(attributePaths = {"author"})
    Optional<Book> findById(Long id);
}
```

`@EntityGraph` 覆蓋預設 fetch 策略，一次載入關聯資料，避免 N+1 查詢問題。方法簽名不變，Service 不用改。

Optional：情境是「查一本不存在的書」

打 `GET /api/books/999`，資料庫裡沒有 999。同一件事，兩種寫法：

❌ 回 null

```
Book book = dao.findByIdOrNull(id);
// book 是 null
return mapper.toResponse(book);
```

NullPointerException → 500
前端看到伺服器爆掉，還要翻 log

✅ 回 Optional

```
Book book = dao.findById(id)
    .orElseThrow(() ->
        new NotFoundException("Book: " + id));
return mapper.toResponse(book);
```

NotFoundException → 404
前端知道是「沒這本書」

重點不是「Optional 比較潮」，是 `null` 可以被忘記檢查、`Optional` 不行——你非得寫 `orElseThrow` 之類的收尾才拿得到 `Book`。

忘記檢查 vs 編譯器逼你檢查

❌ 每個呼叫端都要自己記得

```java
Book book = dao.findByIdOrNull(id);
if (book == null) {            // 漏寫就是 500
    throw new NotFoundException("...");
}
return book.getTitle();
```

✅ 漏寫就編不過

```java
// Optional<Book> 沒有 getTitle()
return dao.findById(id)
    .map(Book::getTitle)       // 有才轉換
    .orElse("(未知書名)");      // 沒有給預設值
```

orElseThrow
沒有就丟例外 → CRUD 用這個

orElse
沒有就給預設值

ifPresent
有才做，沒有就跳過

`BaseServiceImpl.getById` 已經是第一種，所以 Book 的 CRUD 你一行都不用寫。

那 author 是 null 呢？

情境換了：書存在，但 `author_id` 沒填。這次**不要**自己用 Optional。

❌ 在 Service 手動拆關聯

```
BookResponse res = mapper.toResponse(book);
String name = Optional
    .ofNullable(book.getAuthor())
    .map(Author::getName)
    .orElse(null);
// 再想辦法塞回 record…（record 不能改）
```

✅ 交給 Mapper 宣告

```
@Mapping(source = "author.name",
         target = "authorName")
BookResponse toResponse(Book book);

// Service 只有這行
return mapper.toResponse(book);
```

為什麼右邊不會 NPE

```
authorName = book.getAuthor() == null ? null : book.getAuthor().getName();
```

MapStruct 產的 code 自己補了 null 檢查。沒作者就是 `authorName: null`，你不用再包一層。

分工：`Optional` 管「這筆資料存不存在」，`@Mapping` 管「欄位怎麼搬」。不是二選一，是用在不同地方。

改欄位：先改 Entity，再 `change_*`

`make:migration` 認檔名：`create_` 建表、`add_` 加欄、`change_` 改型別。

`change_*` 出 `MODIFY COLUMN`，**新型別只讀 Entity**。找不到那個欄位就失敗、不寫檔——猜錯的 MODIFY 會截斷既有資料。

課堂自己加一個用不到的欄，專門看遷移長什麼：

```
@Column(name = "stock")
private Integer stock;   // 加在 Book.java，DTO 不用動
```

```
birc make:migration add_stock_to_book_table
# 打開 SQL：ADD COLUMN stock INT NULL
birc migrate

# 把 Integer 改成 Long，再：
birc make:migration change_stock_on_book_table
# 打開 SQL：MODIFY COLUMN stock BIGINT NULL
birc migrate
```

故意打 `change_foo_on_book_table`：Entity 沒有 `foo`，應失敗、不寫檔。

軟刪：`--soft-delete`

硬刪是 `DELETE FROM`。軟刪是多一欄 `deleted_at`：刪的時候填時間戳，列還在，查詢當它不存在。

`make` 時加旗標，Entity 跟 SQL 一次到位：

```
birc make Review --fields content:String --migration --soft-delete --public
birc migrate
```

Entity 會有 Hibernate `@SoftDelete(columnName = "deleted_at", strategy = TIMESTAMP)`。建表 SQL 多 `deleted_at TIMESTAMP NULL`。`deleteById` 不用改。

沒有 restore / withTrashed。之後要救回來得自己寫。

不要對已有表跑 `add_deleted_at_to_book_table`：那不是 Entity 欄位，產生器會退回 VARCHAR(50)。軟刪請在 make 時加旗標。

課堂：刪一筆 Review，再看 SQL

`PUBLIC_PATHS` 放行 `/api/reviews`、`/api/reviews/**`，重啟 `bootRun`：

```
curl -s -X POST http://localhost:8080/api/reviews \
  -H 'Content-Type: application/json' \
  -d '{"content":"好看"}'
curl -s http://localhost:8080/api/reviews
curl -s -X DELETE http://localhost:8080/api/reviews/1
curl -s http://localhost:8080/api/reviews      # 空陣列
curl -s http://localhost:8080/api/reviews/1    # 404
```

列還在，只是 API 當它沒了：

```
docker compose exec db mysql -u"$DB_USER" -p"$DB_PASSWORD" "$DB_DATABASE" \
  -e "SELECT id, content, deleted_at FROM review;"
```

`deleted_at` 有值。GET 是 404，不是因為列被清掉。

登入：`birc add auth`

現在我們來加入登入功能，birc-generator提供了快速加入登入模組的方法。

再加入前我們可以先做git儲存

```
git init
git add .
git commit -m "init"
```

接著加入 auth 模組

```
birc add auth
```

可以看到新增了檔些檔案。

登入：`birc add auth`

產生假資料

```
birc make:seeder AuthUser
```

再 `AuthUserSeeder.java` 加入這三行，密碼沒有人在明文儲存的

```
package tw.edu.ntub.birc.bookstore.seeder;

import lombok.RequiredArgsConstructor;

import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Component;
import tw.edu.ntub.birc.bookstore.databaseconfig.dao.AuthUserDAO;
import tw.edu.ntub.birc.bookstore.databaseconfig.entity.AuthUser;

@Component
@RequiredArgsConstructor
public class AuthUserSeeder implements Seeder {

    private final AuthUserDAO authUserDAO;
    private final PasswordEncoder passwordEncoder;

    @Override
```

登入：`birc add auth`

測試

```
curl -i -X POST http://localhost:8080/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"account":"tester","password":"secret"}'
curl -s http://localhost:8080/api/auth/me -H "Authorization: Bearer $TOKEN"
```

會生什麼
auth_users 、 /api/auth/login 、 /api/auth/me、JWT filter

Token 在標頭
成功看 `X-Auth-Token` ，body 只回帳號與權限

之後帶 Bearer
沒帶 → 401。權限不夠 → 403

`JWT_SECRET` 寫在 `.env` 。

權限：`birc add permission`

能成功登入之後，就是權限的部分了，相信各位都知道什麼時候要回401，什麼時候要回403

```
birc add permission
```

會生 `RequirePermission.java`、`PermissionAspect.java`、`SecurityUtils.java`，gradle 接 `spring-boot-starter-aspectj`。

產生資料表 auth_roles 的假資料

現在來產生權限角色的資料

```
birc make:seeder AuthRole
```

```
AuthRole user = new AuthRole();
user.setPosition("ROLE_USER");
authRoleDAO.save(user);

AuthRole admin = new AuthRole();
admin.setPosition("ROLE_ADMIN");
authRoleDAO.save(admin);
```

執行 seed 指令來產生資料到 DB 裡

權限：`birc add permission`

使用方式 有提供有很多種方式，越複雜的專案越會使用 `SecurityUtils` 這樣的功具來管理大量的角色權限

```
// 單一權限
@RequirePermission("ROLE_ADMIN")
// 多重權限（任一符合即可，等同 hasAnyAuthority）
@RequirePermission({"ROLE_ADMIN", "ROLE_AUDITOR"})
// 搭配 SecurityUtils 常用組合
@RequirePermission(SecurityUtils.HAS_SYS_ADMIN_AUTHORITY)
```

在 Controller 使用

那我們現在回到Book的Controller.java 先簡單的為刪除api 加上權限的驗證

```
@RequirePermission("ROLE_ADMIN")
@DeleteMapping("/{id}")
public Result<Void> delete(@PathVariable Long id) { ... }
```

課堂：刪書要有 admin

先 POST 一本當砲灰，用回傳的 id（不要刪 seed 的 1）：

```
# 1. 沒帶 token → 403 缺少權限
curl -i -X DELETE http://localhost:8080/api/books/$ID

# 2. tester 沒有 ROLE_ADMIN，帶 JWT 仍 403
# 從 X-Auth-Token 抄到 TOKEN=
curl -si -X POST http://localhost:8080/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"account":"tester","password":"secret"}'
curl -i -X DELETE http://localhost:8080/api/books/$ID \
  -H "Authorization: Bearer $TOKEN"
```

舊 JWT 的 claim 不會跟著 UPDATE 變，一定要再登入一次。

SecurityUtils 常用權限組合

```
// 系統管理員或一般管理員
public static final String HAS_SYS_ADMIN_AUTHORITY =
    "hasAnyAuthority('ROLE_SYS_ADMIN', 'ROLE_ADMIN')";
// 管理員 + 承辦人
public static final String HAS_ADMIN_AND_HANDLER_AUTHORITY =
    "hasAnyAuthority('ROLE_SYS_ADMIN', 'ROLE_ADMIN', 'ROLE_AUDITOR')";
```

PART 04

ValidGroup 與 Spotless

create --yes 預設勾這兩個，Book 的驗證跟排版都靠它們

ValidGroup：同一份 DTO，兩套必填

Bean Validation 的 `groups` 決定「這次請求要跑哪些註解」。專案裡是空的 marker interface，檔在 `validation/ValidGroup.java`。

組	何時用
`Create`	POST 新增
`Update`	PUT 更新
`Delete`	刪除還要帶 body（原因、確認碼、version）
`Submit`	送審，比草稿嚴

只刪 id 的 `DELETE /api/books/1` 不用 Delete 這組。Submit 留給之後「草稿可少填、送審要全填」。

接到 Book 的兩端必須同一組

Controller：

```
@PostMapping
public Result<BookResponse> create(
        @Validated(ValidGroup.Create.class) @RequestBody BookCreateRequest request)

@PutMapping("/{id}")
public Result<BookResponse> update(
        @PathVariable Long id,
        @Validated(ValidGroup.Update.class) @RequestBody BookCreateRequest request)
```

DTO：

```
public record BookCreateRequest(
        @NotNull(groups = {ValidGroup.Create.class, ValidGroup.Update.class})
        String title,
        // isbn, price, publishedAt 也一樣
) {}
```

要用 `@Validated`，不是 `@Valid`。`@Valid` 不吃 groups。

為什麼不全部裸寫 @NotNull？

骨架現在 Create / Update 都必填，是為了能馬上打 CRUD。之後若更新允許只改書名：

```
public record BookCreateRequest(
        @NotNull(groups = ValidGroup.Create.class)              // 只有新增必填
        String isbn,

        @NotNull(groups = {ValidGroup.Create.class, ValidGroup.Update.class})
        String title                                            // 兩邊都必填
) {}
```

POST 缺 isbn → 400。PUT 可以不帶 isbn，title 仍必填。還是同一份 `BookCreateRequest`，不必拆成兩份 DTO。

`Delete` / `Submit` 今天是空的，不要刪。預設 CRUD 也不要標這兩組。

課堂驗驗證

缺 `title` 打一次：

```
curl -i -X POST http://localhost:8080/api/books \
  -H 'Content-Type: application/json' \
  -d '{"isbn":"9780321125217","price":1800,"publishedAt":"2003-08-30"}'
```

應是 400。這條不會進 Service。

`ExceptionHandleController` 只攔 `ProjectException`。驗證失敗走 Spring 預設的 `MethodArgumentNotValidException`，之後要統一 JSON 再自己加 `@ExceptionHandler`。

Spotless：只管格式，不改邏輯

`--yes` 會在專案根放 `spotless_formatter.xml`（Eclipse 4.31），Gradle 接 Spotless plugin。

掃這些 Java：

- `controller/`
- `service/**`
- `modules/**/entity/**`
- `modules/**/dao/`

會做：依 xml 排版、清沒用的 import、去行尾空白、檔案結尾換行。

不管命名、不管潛在 bug。不要再另裝 Checkstyle 跟它搶排版。

兩個指令

```
./gradlew spotlessApply    # 直接改檔
./gradlew spotlessCheck    # 只檢查，CI 用這個
```

Windows：`.\gradlew.bat spotlessApply`

習慣

commit 前跑 `spotlessApply`，再看 diff。

CI 跑 `spotlessCheck`，不要在 CI 裡 Apply（會改檔）。

3. 三種啟動，選一個

`create` 預設勾 Docker，會有兩份 compose。**這堂課用第一種。**

每天寫程式

```
docker compose up -d db
./gradlew bootRun
```

只起 MySQL。App 在本機，改完重啟、能 debug。容器裡不跑 Gradle。

本機整包容器

```
docker compose up
```

讀 `docker-compose.yml`。容器裡 **build** App（慢）、db 對外映射、開 JDWP 5005。給不會裝 Java 的人，或要測「跟容器長一樣」。

測試機 / 正式機

```
docker compose -f docker-
compose.prod.yml up -d
```

從 Harbor **pull** 映像，不在機器上 build。db 不對外開 port。CI 只改 `APP_TAG`。

沒加 `-f` 就是那份沒有 `.prod` 的。課堂不要 `docker compose up` 把 app 也建進去，第一次會卡很久。

4. Spring Profiles：dev / prod

產生器**沒有**生 `application-dev.yml` / `application-prod.yml`。共用設定在一份 `application.yml`，用環境變數換值。

切換靠環境變數：

方式	例子
compose（本機）	`SPRING_PROFILES_ACTIVE: dev`
compose（正式）	`SPRING_PROFILES_ACTIVE: prod`
本機 bootRun	`export SPRING_PROFILES_ACTIVE=dev`
啟動參數	`java -jar app.jar --spring.profiles.active=prod`

之後若要依 profile 關 Swagger、改 log 等級，再自己加 `application-prod.yml`。profile 檔會覆蓋共用檔。

本機 vs 正式，對照記

	本機 bootRun	compose prod
DB host	`127.0.0.1`（.env 的 DB_URL）	`db`（compose 服務名）
profile	你 export 的，或預設	`prod`
密碼	`.env`，create 時隨機產生	同一個鍵，值自己換掉
ddl	Flyway + `validate`	一樣，不要改成 `update`

正式環境不要把密碼寫進 yml，走環境變數。

Harbor：`birc add gitlab-ci`

實驗室的 CI **都會把映像推到 Harbor**。沒有 `birc harbor`：Harbor 是倉庫，`birc add gitlab-ci` 生的是 `.gitlab-ci.yml`。

```
birc add gitlab-ci
```

模板帶進**這個專案**的名字，不是寫死 teaching-platform。bookstore 會變成：

寫進 yml 的	值
`IMAGE_NAME`	`$HARBOR_URL/bookstore/bookstore_app`
`DEPLOY_DIR`	`/opt/bookstore/backend`

Harbor 位址、帳密、要部哪台，全部是 GitLab CI/CD Variables（`HARBOR_*`、`BETA_*`、`ONLINE_*`）。不要寫進檔，也不要用 Docker build-arg。

會問 GitLab 專案路徑（CI 只在那個 repo 跑），可留空。`SSH_HOST_KEY` 用 `ssh-keyscan -t ed25519 <host>`，沒填 pipeline 直接失敗。

兩個環境，兩條流水線

映像名：`$HARBOR_URL/bookstore/bookstore_app`（專案名會換）。伺服器目錄 `/opt/bookstore/backend`。

分支	會跑什麼
`development` / MR	`build-beta` → `deploy-beta`（tag `beta-` + 短 sha）
`main`	`build-online` ; `deploy-online` **要手動按**

CI 只改伺服器 `.env` 的 `APP_TAG`。第一次部署要自己先填：

```
DOCKER_IMAGE=harbor.xxx/bookstore/bookstore_app
APP_TAG=latest
```

之後 `docker compose -f docker-compose.prod.yml pull app && up -d --no-deps app`。db 沒起來才會順便起。

這堂不必真的 push。之後作業／專題的 CI 也是這條路：GitLab 建映像 → push Harbor → 機器 pull。

Sentry：`birc add sentry`

```
birc add sentry
```

會生 `SentryConfig` 跟 yml。專案開在 sentry.ntubimdbirc.tw，DSN 從那邊複製，放 `.env`，空白 = 不送：

```
SENTRY_DSN=https://xxxx@sentry.ntubimdbirc.tw/1
```

本機 `source .env` 再重啟。正式機同一鍵寫在伺服器 `.env`（compose prod 已經接了 `SENTRY_DSN`）。

會送
沒接住的 5xx、真的爆掉的例外

故意不送
4xx、`NotFoundException`、驗證失敗。否則每次打錯 API 都告警

過濾寫在 `BeforeSendCallback`：`ProjectException` 看 `getHttpStatus()` 是不是 4xx。之後自己加例外，不必改這份設定。

課堂驗：暫時在 Controller 丟一個 `new RuntimeException("sentry-test")`，重整後 Sentry 要看得到；`GET /api/books/999` 的 404 不該出現。

PART 06

CORS 怎麼設

只改一個環境變數，不要去加 addCorsMappings

為什麼前端會覺得「後端沒開 CORS」

瀏覽器跨 Origin 會先發 OPTIONS preflight。

這次改版把 CORS 掛在 **Spring Security 的 filter chain**：

```
1   http.csrf(csrf -> csrf.disable())
2       .cors(Customizer.withDefaults())
```

只在 `WebMvcConfigurer.addCorsMappings` 登記時，Security 會先擋掉 OPTIONS（401），瀏覽器永遠看不到 `Access-Control-*`。

> 空白 = 不開放跨來源。本機 curl 不受 CORS 影響；Vite `localhost:5173` 會。

正確改法

`.env`：

```
CORS_ALLOWED_ORIGIN_PATTERNS=http://localhost:5173
```

多個前端用逗號：

```
CORS_ALLOWED_ORIGIN_PATTERNS=http://localhost:5173,https://bookstore.example.edu.tw
```

規則：

- 不要尾斜線（`http://localhost:5173/` 不行）
- 不要 `*` / `http://*` / `https://*`（啟動直接失敗）
- 改完要**重啟** Spring，bootRun 不會熱重載 .env

然後重新 `source .env` 再 `./gradlew bootRun`。

對應到程式

```java
@Bean
public CorsConfigurationSource corsConfigurationSource(
        @Value("${app.cors.allowed-origin-patterns:}") String allowedOriginPatterns) {
    // split、trim、去掉尾斜線
    // 是 * 就 throw IllegalStateException
    config.setAllowedOriginPatterns(patterns);
    config.setAllowCredentials(!patterns.isEmpty());
}
```

compose 已經把同一變數傳進容器：

```yaml
environment:
  CORS_ALLOWED_ORIGIN_PATTERNS: ${CORS_ALLOWED_ORIGIN_PATTERNS:-}
```

dev 跟 prod 換的是 `.env` 的值，不是改 Java。

課堂驗 CORS

用瀏覽器或前端 fetch，不要只用 curl（curl 不跑 CORS）。

1. `.env` 空白 → 瀏覽器預檢失敗
2. 設成 `http://localhost:5173` → 該 Origin 過
3. 設成 `*` → 應用程式啟動失敗

> 作業如果接 Vue，Origin 填 Vite 的那個，不要填 API 自己的 8080。

今日路線回顧

npm → birc
npm i -g birc-generator

create --yes
docker、ValidGroup、Spotless

make → migrate → seed
make 一次產檔；進 DB 是 migrate 建表、seed 塞列

env 切環境
Profiles 跟 CORS 都走 .env

下課前確認：POST /api/books 有 id 回來，GET 看得到，DELETE 再 GET 是 404。

指令速查

```
npm i -g birc-generator
birc create bookstore --yes && cd bookstore
docker compose up -d db
set -a && source .env && set +a

birc make Book --fields title:String,isbn:String,price:BigDecimal,publishedAt:LocalDate \
  --migration --seed
# 改 PUBLIC_PATHS（books / files）→ 改 CORS
birc migrate
birc seed
./gradlew bootRun
# 之後：birc add file-upload，再放行 /api/files/**
./gradlew spotlessApply
```

之後要加模組：`birc add auth`、`birc add permission`、`birc add file-upload`、`birc add gitlab-ci`（Harbor）、`birc add sentry`。`--yes` 已經有 OpenAPI，不必再 add。

進階：`birc db:wipe` 只跑 `flywayClean`，把 schema 清掉不重建；要清掉再建回來才是 `birc migrate:reset --force`。

Bookstore is open.

下一步：Spring Profiles 文章，把 dev/prod 再拆細

```
$ birc make Book --fields ... --migration --seed
create → make → migrate → seed → bootRun
```